

Technical Specification:

Territory Management Solution

Prepared by: Aysun Bilgic

Date: 8/2/2025

Table of Contents

1. Objective
2. Solution Overview
3. Key Components
 - 3.1 Custom Metadata Type: Territory_Mapping__mdt
 - 3.2 Apex Trigger: AccountTrigger
 - 3.3 Apex Handler Class: AccountTerritoryHandler
4. Detailed Logic Flow
5. User Interface
6. Supporting Details
7. Testing Strategy
 - 7.1 Manual Testing
 - 7.2 Automated Tests
8. Security & Access Control
9. Error Handling & Logging
10. Benefits & Future-Proofing
11. Potential Enhancements
12. Challenges Encountered and Solutions
- 13. Appendix**
 - 13.1 Trigger Snippet
 - 13.2 Apex Handler Core Logic (Simplified)
 - 13.3 Apex Handler Test Logic
14. References

1. Objective

Automate territory assignment on Salesforce Accounts based on configurable business-defined territory rules using ZIP code and state information.. The system must:

- Assign territory on Account creation and update.
- Trigger assignment when Billing Zip is populated or changed.
- Support strategic zip code mapping for New York and California.
- Include Great Lakes states as a territory.
- Assign "Unassigned" territory if the location is outside defined areas.

- If neither Billing Zip Code nor Billing State is populated, assign the Unknown territory.
- Prepare for future expansion to map all US zip codes (~50,000) without code changes.
- Leverage Apex coding per challenge requirement.
- Enable easy annual map redraws without deployments.

2. Solution Overview

The solution leverages:

- **Custom Metadata Type (CMDT)** for defining zip and state to territory mappings.
- **Apex Trigger and Handler Class** to assign the territory field dynamically.
- Bulkified logic optimized with Maps to avoid nested loops and maintain performance.
- Trigger logic running on Account insert and update selectively updating only when billing postal code fields change.

3. Key Components

3.1 Custom Metadata Type: Territory_Mapping__mdt

- **Fields:**
 - o Territory__c (Picklist/Text): Name of the territory (e.g., “New York - Strategic”).
 - o State__c (Text): State code (e.g., NY, CA, IL)—optional for state-wide rules.
 - o Zip_Code__c (Text): Postal code (e.g., 10002)—optional for specific zip code rules.
 - o Priority__c (Number): Optional for future use to resolve overlapping rules.
- **Sample Records:**

Name	Territory__c	State__c	Zip_Code__c	Priority__c
NY Strategic 10002	New York - Strategic	NY	10002	1
NY General	New York	NY		2
CA Strategic 90210	California - Strategic	CA	90210	1
CA General	California	CA		2

3.2 Apex Trigger: AccountTrigger

- Fires **before insert** and **before update** on Account.
- Calls Apex handler to assign territory on:
 - o All new Accounts (insert).
 - o Updated Accounts only if BillingPostalCode is newly populated or changed.

3.3 Apex Handler Class: AccountTerritoryHandler

- Queries CMDT records.
- Separates zip-based and state-based mappings into Maps for O(1) lookups.
- Processes Accounts in bulk, assigning territory based on:
 - o Zip code lookup priority.
 - o State code lookup fallback.
 - o “Unknown” if both zip and state are blank.
 - o “Unassigned” otherwise.
- Exception handling surrounds logic to catch unexpected errors.

4. Detailed Logic Flow

1. Upon Account creation or update, the trigger checks context:
 - For inserts: all new Accounts are processed.
 - For updates: only those whose billing zip code changed.
2. The handler class queries current territory mappings from Territory_Mapping__mdt.
3. It builds two Maps:
 - zipToMapping: zip code → territory mapping.
 - stateToMapping: state code → territory mapping.
4. For each Account:

- If zip code matches a strategic zip in zipToMapping, assign corresponding territory.
- Else if state code matches in stateToMapping, assign corresponding territory.
- Else if neither billing zip nor state is populated, assign "Unknown".
- Else assign "Unassigned".

5. The Territory__c field on Account is updated accordingly.

5. User Interface

- The Territory__c field is visible and editable on the Account page layout, allowing users and admins to view and update territory assignments directly on Account records.
- The Territory__c, BillingState, and BillingPostalCode fields are included in all Account list views. This supports territory assignment logic verification, facilitates change tracking, and enables users to monitor accounts by territory and location attributes efficiently.
- These fields may be surfaced in a future Experience Cloud portal to provide regional managers with visibility into account assignments within their territories, enhancing regional oversight and sales performance tracking.

6. Supporting Details

- The Territory__c field is a custom picklist on Account with values:
 - New York - Strategic
 - New York
 - California - Strategic
 - California
 - Great Lakes
 - Unassigned
 - Unknown
- The solution uses **bulkified Apex** and avoids nested for loops by using Maps.
- Custom Metadata enables **admin-friendly management** of territory definitions without Apex code changes or deployments.

7. Testing Strategy

7.1 Manual Testing

- Create Accounts via Salesforce UI covering all test cases:
 - Strategic zip codes.
 - General state codes.
 - Unknown/missing addresses.
 - Unassigned states.
- Modify BillingPostalCode and BillingState on updates; confirm territory updates accordingly.

7.2 Automated Tests

- Apex test class AccountTerritoryHandlerTest:
 - Inserts sample Accounts matching each territory condition.
 - Checks that Territory__c is assigned correctly on insert.
 - Updates Accounts' BillingPostalCode and confirms territory updates.
 - Validates trigger firing only when relevant fields change.
 - Covers both positive and negative scenarios for territory determination.
 -

8. Security & Access Control

No custom permission set or profile changes were made beyond the default configuration.

9. Error Handling & Logging

- All Apex logic is implemented with try-catch blocks to ensure safe execution and graceful error handling. This approach prevents unhandled exceptions from failing transactions and allows for controlled response to unexpected issues.
- Currently, errors and exceptions are logged using System.debug() statements in lower environments (such as sandboxes and developer orgs). This enables developers to trace and diagnose issues during development and testing phases without impacting production performance.

10. Benefits & Future-Proofing

- **Scalable:** Supports expansion to all 50,000 U.S. zip codes via CMDT records.
- **Maintainable:** Admins can redraw territories annually by updating Custom Metadata no developer needed.
- **Bulkified and Performant:** Uses Maps for fast lookups and minimizes trigger processing.
- **Extensible:** Priority__c field can be used to manage overlapping rules in the future.
- **Compliant with Salesforce Best Practices:** Clear separation of trigger and handler logic, proper bulkification, and meaningful comments.

11. Potential Enhancements

- **Extend mapping to cover additional address fields.**
Add a Country__c field—preferably as a Picklist for standardized ISO codes, or as Text for flexibility—to the CMDT to support territory assignment by country and international address logic.
- **Update mapping logic for field types to support both Text and Picklist field types.**
Ensures compatibility with various Salesforce org configurations, including the standard State and Country/Territory Picklists.
- **Use Custom Metadata relationships or hierarchies for complex territory structures.**
Enables scalable and maintainable multi-level territory management by defining parent-child relationships within CMDT records.
- **Integrate with Salesforce Territory Management (STM).**
To leverage advanced features such as role-based access control, sharing rules, and territory forecasting.
- **Add scheduled Batch Apex jobs or flows to handle mass territory revisions.**
Examples include annual redraws or large-scale geographic changes, improving operational efficiency and ensuring territory assignments remain up-to-date at scale.
- **Refer to Salesforce documentation on State and Country/Territory Picklists.**
For best practices when configuring these fields to ensure data consistency and reporting accuracy.
- **Use a Global Picklist Value Set for Territory values.**
Define territory picklist values centrally using a Global Picklist Value Set to ensure consistency across multiple objects and fields. This reduces errors, simplifies maintenance, and supports scalable territory management.

- **Introduce a custom error logging object for improved error handling and monitoring.**

This object will centralize error records, allow querying and reporting on errors, and enable proactive monitoring of system health and issues supporting comprehensive audit tracking and operational transparency.

12.Challenges Encountered and Solutions

1. Selecting the Right Data Structure for Territory Logic

At first, I used a custom object plus two separate Custom Metadata Types (CMDTs), one for states and one for zip codes which led to unclear logic and inconsistent territory assignments.

Solution: Consolidated state and zip code info into a single CMDT with fields for both. Modified Apex to prioritize zip code matches, falling back to state matches only if needed. This simplified logic, reduced metadata maintenance, and eliminated the need for extra DML operations.

2. Zip Code vs. State Mapping Conflict – Duplicate Territory Results

Initial logic didn't separate zip code and state mappings properly, causing many Accounts to default to the wrong territory (New York – Strategic).

Solution: Created two distinct Maps, zip code and state, and made zip-based mapping take precedence. Corrected CMDT records to ensure accurate territory references. This prevented overlaps and aligned assignments with business rules.

3. Trigger Optimization Based on Field Changes

To avoid unnecessary processing and improve efficiency, the trigger now fires territory assignment logic only when the BillingPostalCode changes, detected by comparing Trigger.oldMap and Trigger.newMap.

This meets business requirements and allows easy extension to monitor BillingState changes for future enhancements.

4. Incorrect Test Data in Account Records Caused Assertion Failures

Test failures occurred due to Accounts referencing zip codes or states not present in metadata mappings.

Fix: Updated test data to use valid zip, state,territory combinations, ensuring tests reflected real-world scenarios and passed reliably.

5. Code Failed When Picklist Fields Were Enabled

The initial code assumed BillingState and Country fields were plain text. However, during the testing phase, Salesforce had State and Country/Territory Picklists enabled by default, causing string comparisons to fail and breaking territory assignment logic.

Fix: Temporarily disabled the State and Country/Territory Picklists in Setup to revert fields to plain text, restoring functionality.

Lesson: Learned that production-ready code must be flexible to accommodate both text and picklist field configurations, ensuring environment-agnostic and sustainable logic.

6. Misconfigured Custom Metadata: New York vs. New York – Strategic

Some Accounts assigned incorrectly due to CMDT misconfiguration where non-strategic zip codes pointed to strategic territory.

Resolution: Corrected CMDT to assign only defined strategic zip codes to “New York – Strategic” and default others to “New York”, aligning territory assignment with business rules.

Summary

Addressing these challenges helped me deliver a robust and maintainable territory assignment solution that:

- Uses consolidated and scalable metadata structures
- Respects priority business rules accurately
- Optimizes trigger performance by minimizing unnecessary executions
- Supports consistent deployments of configurations and data
- Handles variable environment setups gracefully
- Ensures valid and reliable testing practices
- Upholds accurate metadata governance for data integrity

This experience reflects a mature understanding of Salesforce architecture and best practices relevant to enterprise territory management.

13. Appendix

13.1 Trigger Snippet

```
trigger AccountTrigger on Account (before insert, before update) {
```



```

if (Trigger.isInsert) {
    AccountTerritoryHandler.assignTerritories(Trigger.new);
}

if (Trigger.isUpdate){
    List<Account> changedZipAccounts = new List<Account>();

    for(Account acc : Trigger.new) {

        Account oldAcc = Trigger.oldMap.get(acc.Id);

        if (acc.BillingPostalCode != oldAcc.BillingPostalCode) {

            changedZipAccounts.add(acc);

        }

    }

    if (!changedZipAccounts.isEmpty()) {

        AccountTerritoryHandler.assignTerritories(changedZipAccounts);

    }

}
}

```

13.2 Apex Handler Core Logic (Simplified)

```

public class AccountTerritoryHandler {

    public static void assignTerritories(List<Account> accts) {

        try {

            //Query all relevant territory mappins

            List<Territory_Mapping__mdt> mappings = [SELECT Territory__c, State__c, Priority__c, Zip_Code__c

                FROM Territory_Mapping__mdt

                ORDER BY Priority__c ASC

            ];

            //Map zip code to mapping record (for strategic zip matching)

            Map<String, Territory_Mapping__mdt> zipToMapping = new Map<String, Territory_Mapping__mdt>(); //new york str + cal str

```

```

//Map state to mapping record (for state matching)

Map<String, Territory_Mapping__mdt> stateToMapping = new Map<String, Territory_Mapping__mdt>();// new york + calif +
great lake

for(Territory_Mapping__mdt m : mappings) {
    if (!String.isBlank(m.Zip_Code__c)) {
        zipToMapping.put(m.Zip_Code__c, m);

    } else if (!String.isBlank(m.State__c))
        stateToMapping.put(m.State__c, m);

}

for (Account acc : accts) {
    String state = acc.BillingState;
    String zip = acc.BillingPostalCode;
    String newTerritory = null;

    if (!String.isBlank(zip) && zipToMapping.containsKey(zip)) {
        newTerritory = zipToMapping.get(zip).Territory__c;
    } else if (!String.isBlank(state) && stateToMapping.containsKey(state)) {
        newTerritory = stateToMapping.get(state).Territory__c;
    } else if (String.isBlank(zip) && String.isBlank(state)) {
        newTerritory = 'Unknown';
    } else {
        newTerritory = 'Unassigned';
    }
}

```

```

        acc.Territory__c = new Territory;
    }

    } catch (Exception e) {

        System.debug('An error occurred in assignTerritories: ' + e.getMessage());

    }

}
}

```

13.3 Apex Handler Test Logic

```

@Test
public class AccountTerritoryHandlerTest {

    @Test
    static void testAssignTerritories() {

        List<Account> accountsToInsert = new List<Account>{

            new Account(Name='NY Strategic Account', BillingPostalCode='10014', BillingState='NY'),
            new Account(Name='CA Strategic Account', BillingPostalCode='90024', BillingState='CA'),
            new Account(Name='CA Account', BillingPostalCode='90756', BillingState='CA'),
            new Account(Name='IL Account', BillingPostalCode='60601', BillingState='IL'),
            new Account(Name='NY State Only Account', BillingPostalCode='12000', BillingState='NY'),
            new Account(Name='TX Unassigned', BillingPostalCode='73301', BillingState='TX'),
            new Account(Name='No Address Account') // Missing ZIP and State

        };

        Test.startTest();

        insert accountsToInsert;

        Test.stopTest();

        Map<String, String> expectedResults = new Map<String, String>{

            'NY Strategic Account' => 'New York - Strategic',

```

```

        'CA Strategic Account' => 'California - Strategic',

        'CA Account' => 'California',

        'IL Account' => 'Great Lakes',

        'NY State Only Account' => 'New York',

        'TX Unassigned' => 'Unassigned',

        'No Address Account' => 'Unknown'

    };

    for (Account acc : [SELECT Name, Territory__c FROM Account WHERE Name LIKE '%Account']) {

        System.assertEquals(expectedResults.get(acc.Name), acc.Territory__c, 'Territory mismatch for: ' + acc.Name);

    }

}

@isTest

static void testUpdateTerritoryAssignmentWhenZipChanges() {

    Account acc = new Account(Name='Update Zip Account', BillingState='NY', BillingPostalCode='10014');

    insert acc;

    Test.startTest();

    acc.BillingState = 'CA';

    update acc;

    acc.BillingPostalCode = '90210';

    update acc;

    Test.stopTest();

    acc = [SELECT Territory__c FROM Account WHERE Id = :acc.Id];

    System.assertEquals('California - Strategic', acc.Territory__c);

}

}

```

14. References

- [State and Country/Territory Picklists](#)

Official Salesforce documentation on configuring and managing standardized state and country picklists for improved data consistency.

- [Custom Metadata Types](#)

Guide to creating and using Custom Metadata Types in Salesforce to drive declarative, admin-friendly configuration.

- [Salesforce Territory Management](#)

Overview of Salesforce's Territory Management features, including structure, assignment rules, and best practices for scalable territory modeling.

-